

Fourier Series Properties – Problem Patterns and Implementation Steps

1. Convention

For a periodic signal with period T_0 ,

$$\omega_0 = \frac{2\pi}{T_0}.$$

The exponential Fourier Series is

$$x(t) = \sum_{k=-\infty}^{\infty} C_k e^{jk\omega_0 t},$$

where

$$C_k = \frac{1}{T_0} \int_{t_0}^{t_0+T_0} x(t) e^{-jk\omega_0 t} dt.$$

The trigonometric form is

$$x(t) = \frac{a_0}{2} + \sum_{k=1}^{\infty} [a_k \cos(k\omega_0 t) + b_k \sin(k\omega_0 t)].$$

The relationships are

$$C_0 = \frac{a_0}{2}, \quad C_k = \frac{1}{2}(a_k - jb_k), \quad k > 0,$$

$$C_{-k} = \frac{1}{2}(a_k + jb_k).$$

2. Generic Numerical FS Implementation

This function directly implements the coefficient definition.

```
def compute_fs_coefficients(t, x, T0, K):
    omega0 = 2 * np.pi / T0

    k_values = np.arange(-K, K + 1)

    C = np.zeros(
        2 * K + 1,
        dtype=complex
    )

    for i, k in enumerate(k_values):
```

```

    exponential = np.exp(
        -1j * k * omega0 * t
    )

    integrand = x * exponential

    C[i] = (
        np.trapezoid(integrand, t)
        / T0
    )

return k_values, C

```

For reconstruction,

$$x_K(t) = \sum_{k=-K}^K C_k e^{jk\omega_0 t}.$$

```

def reconstruct_fs(t, k_values, C, T0):
    omega0 = 2 * np.pi / T0

    x_rec = np.zeros(
        len(t),
        dtype=complex
    )

    for k, Ck in zip(k_values, C):
        x_rec += (
            Ck * np.exp(1j * k * omega0 * t)
        )

    return x_rec

```

3. Master List of Properties

Property	Time-domain operation	FS coefficient result
Linearity	$ax(t) + by(t)$	$aC_k + bD_k$
Time shift	$x(t - t_0)$	$C_k e^{-jk\omega_0 t_0}$
Time reversal	$x(-t)$	C_{-k}
Conjugation	$x^*(t)$	C_{-k}^*
Real signal	$x(t) \in \mathbb{R}$	$C_{-k} = C_k^*$
Differentiation	$x'(t)$	$jk\omega_0 C_k$
Second derivative	$x''(t)$	$-(k\omega_0)^2 C_k$
Integration	$\int x(t) dt$	$C_k / (jk\omega_0)$ for $k \neq 0$
Complex modulation	$x(t) e^{jm\omega_0 t}$	C_{k-m}
Cosine modulation	$x(t) \cos(m\omega_0 t)$	$(C_{k-m} + C_{k+m})/2$
Sine modulation	$x(t) \sin(m\omega_0 t)$	$(C_{k-m} - C_{k+m})/(2j)$
Time multiplication	$x(t)y(t)$	discrete convolution of coefficients
Periodic convolution	$x *_p y$	$C_k D_k$ under normalized definition
Parseval	average power	$\sum_k C_k ^2$
DC component	average value	C_0

1 4. Linearity

If

$$z(t) = ax(t) + by(t),$$

and $x(t) \leftrightarrow C_k$, $y(t) \leftrightarrow D_k$, then

$$Z_k = aC_k + bD_k.$$

Implementation:

```
z = a * x + b * y

k, Z = compute_fs_coefficients(t, z, T0, K)
_, C = compute_fs_coefficients(t, x, T0, K)
_, D = compute_fs_coefficients(t, y, T0, K)

Z_theory = a * C + b * D
```

Compare Z and Z_{theory} .

2 5. Time Shift

For

$$y(t) = x(t - t_0),$$

$$Y_k = C_k e^{-jk\omega_0 t_0}.$$

Magnitude is unchanged:

$$|Y_k| = |C_k|,$$

while phase changes by

$$-k\omega_0 t_0.$$

Implementation:

```
y = signal_function(t - t0)

k, Y = compute_fs_coefficients(t, y, T0, K)
_, C = compute_fs_coefficients(t, x, T0, K)

Y_theory = C * np.exp(
    -1j * k * omega0 * t0
)
```

This is a very likely verification problem.

3 6. Time Reversal

For

$$y(t) = x(-t),$$

$$Y_k = C_{-k}.$$

If coefficients are stored in the order $[-K, \dots, K]$, the theoretical array is the reversed array.

```
y = signal_function(-t)

k, Y = compute_fs_coefficients(t, y, T0, K)
_, C = compute_fs_coefficients(t, x, T0, K)

Y_theory = C[::-1]
```

4 7. Complex Conjugation

For

$$y(t) = x^*(t),$$

$$Y_k = C_{-k}^*.$$

Implementation:

```
y = np.conj(x)

k, Y = compute_fs_coefficients(t, y, T0, K)
_, C = compute_fs_coefficients(t, x, T0, K)

Y_theory = np.conj(C[::-1])
```

5 8. Real-Signal Conjugate Symmetry

For real $x(t)$,

$$C_{-k} = C_k^*.$$

Therefore,

$$|C_{-k}| = |C_k|$$

and the phase is antisymmetric, apart from phase wrapping.

Implementation:

```
k, C = compute_fs_coefficients(t, x, T0, K)

C_theory = np.conj(C[::-1])

error = np.max(
    np.abs(C - C_theory)
)
```

6 9. Even and Odd Symmetry

For a real even signal,

$$x(t) = x(-t),$$

the coefficients are real and even:

$$\boxed{\text{Im}(C_k) \approx 0}.$$

For a real odd signal,

$$x(t) = -x(-t),$$

the coefficients are imaginary and odd:

$$\boxed{\text{Re}(C_k) \approx 0}.$$

This is often tested by calculating the coefficient array and checking the maximum unwanted real/imaginary component.

```
k, C = compute_fs_coefficients(t, x, T0, K)

max_real = np.max(np.abs(np.real(C)))
max_imag = np.max(np.abs(np.imag(C)))
```

7 10. Differentiation

If

$$y(t) = \frac{dx(t)}{dt},$$

then

$$\boxed{Y_k = jk\omega_0 C_k}.$$

Implementation:

```
dx = analytical_derivative(t)

k, Y = compute_fs_coefficients(t, dx, T0, K)
_, C = compute_fs_coefficients(t, x, T0, K)

Y_theory = 1j * k * omega0 * C
```

If an analytical derivative is unavailable, `np.gradient(x,t)` can be used, but it introduces additional numerical error.

8 11. Second Derivative

$$\boxed{x''(t) \leftrightarrow -(k\omega_0)^2 C_k}.$$

Implementation:

```

d2x = analytical_second_derivative(t)

k, Y = compute_fs_coefficients(t, d2x, T0, K)
_, C = compute_fs_coefficients(t, x, T0, K)

Y_theory = -(k * omega0)**2 * C

```

9 12. Integration

For a periodic antiderivative, the DC coefficient must be zero. For $k \neq 0$,

$$Y_k = \frac{C_k}{jk\omega_0}.$$

Implementation:

```

Y_theory = np.zeros_like(C)

mask = k != 0

Y_theory[mask] = (
    C[mask]
    / (1j * k[mask] * omega0)
)

```

The $k = 0$ term must be treated separately. If $C_0 \neq 0$, the ordinary integral contains a ramp and is not periodic.

10 13. Complex Exponential Modulation

For integer m ,

$$y(t) = x(t)e^{jm\omega_0 t},$$

then

$$Y_k = C_{k-m}.$$

Implementation requires coefficient-index shifting.

```

C_dict = {
    int(ki): Ci
    for ki, Ci in zip(k, C)
}

Y_theory = np.zeros_like(C)

for i, ki in enumerate(k):
    source = int(ki - m)

    if source in C_dict:
        Y_theory[i] = C_dict[source]

```

Be careful at the boundaries because coefficients outside $[-K, K]$ were not calculated.

11 14. Cosine Modulation

For

$$y(t) = x(t) \cos(m\omega_0 t),$$

$$Y_k = \frac{1}{2} [C_{k-m} + C_{k+m}].$$

Implementation:

```
y = x * np.cos(m * omega0 * t)

k, Y = compute_fs_coefficients(t, y, T0, K)
_, C = compute_fs_coefficients(t, x, T0, K)

C_dict = {
    int(ki): Ci
    for ki, Ci in zip(k, C)
}

Y_theory = np.zeros_like(C)

for i, ki in enumerate(k):
    c1 = C_dict.get(int(ki - m), 0)
    c2 = C_dict.get(int(ki + m), 0)

    Y_theory[i] = 0.5 * (c1 + c2)
```

This is one of the most useful Fourier Series property problems to practice.

12 15. Sine Modulation

For

$$y(t) = x(t) \sin(m\omega_0 t),$$

$$Y_k = \frac{C_{k-m} - C_{k+m}}{2j}.$$

Implementation:

```
y = x * np.sin(m * omega0 * t)

k, Y = compute_fs_coefficients(t, y, T0, K)
_, C = compute_fs_coefficients(t, x, T0, K)

C_dict = {
    int(ki): Ci
    for ki, Ci in zip(k, C)
}

Y_theory = np.zeros_like(C)

for i, ki in enumerate(k):
```

```

c1 = C_dict.get(int(ki - m), 0)
c2 = C_dict.get(int(ki + m), 0)

Y_theory[i] = (c1 - c2) / (2j)

```

13 16. Multiplication of Two Periodic Signals

If

$$z(t) = x(t)y(t),$$

then the Fourier Series coefficients are the discrete convolution

$$Z_k = \sum_{m=-\infty}^{\infty} C_m D_{k-m}.$$

Thus

multiplication in time $\leftrightarrow$ convolution of FS coefficients.

The direct route is:

```

z = x * y

k, Z = compute_fs_coefficients(t, z, T0, K)

```

The theoretical coefficient sequence can be calculated with a nested sum:

```

D_dict = {
    int(ki): Di
    for ki, Di in zip(k, D)
}

Z_theory = np.zeros_like(C)

for i, ki in enumerate(k):

    total = 0j

    for j, mj in enumerate(k):

        target = int(ki - mj)

        if target in D_dict:
            total += C[j] * D_dict[target]

    Z_theory[i] = total

```

14 17. Periodic Convolution

Define normalized periodic convolution as

$$z(t) = \frac{1}{T_0} \int_{t_0}^{t_0+T_0} x(\tau)y(t-\tau)d\tau.$$

Then

$$Z_k = C_k D_k.$$

This gives the useful duality:

$$\text{time multiplication} \leftrightarrow \text{coefficient convolution}$$

and

$$\text{normalized periodic convolution} \leftrightarrow \text{coefficient multiplication.}$$

A coding problem can therefore numerically calculate the periodic convolution, calculate its FS coefficients, and compare them with

$$C_k D_k.$$

15 18. Parseval's Theorem

For the exponential Fourier Series,

$$\frac{1}{T_0} \int_{t_0}^{t_0+T_0} |x(t)|^2 dt = \sum_{k=-\infty}^{\infty} |C_k|^2.$$

Numerically:

```
k, C = compute_fs_coefficients(t, x, T0, K)

power_time = (
    np.trapezoid(np.abs(x)**2, t)
    / T0
)

power_coefficients = np.sum(
    np.abs(C)**2
)

error = abs(
    power_time - power_coefficients
)
```

Because only $2K+1$ coefficients are retained, the values may not be exactly equal. Increasing K generally improves the approximation.

16 19. DC Component

The zeroth coefficient is the average value:

$$C_0 = \frac{1}{T_0} \int_{t_0}^{t_0+T_0} x(t) dt.$$

A simple verification problem is:

```
k, C = compute_fs_coefficients(t, x, T0, K)

C0 = C[k == 0][0]
```

```

C0_direct = (
    np.trapezoid(x, t)
    / T0
)

error = abs(C0 - C0_direct)

```

17 20. Fourier Series Reconstruction

Given C_k , reconstruct

$$x_K(t) = \sum_{k=-K}^K C_k e^{jk\omega_0 t}.$$

Implementation:

```

K = 20

k, C = compute_fs_coefficients(
    t, x, T0, K
)

x_rec = reconstruct_fs(
    t, k, C, T0
)

mse = np.mean(
    (x - np.real(x_rec))**2
)

```

A typical problem asks you to compare reconstructions for several values of K .

18 21. Gibbs Phenomenon

For a signal with discontinuities, a finite Fourier Series reconstruction oscillates near the discontinuity.

A useful experiment is to reconstruct a square wave using

$$K = 5, \quad 20, \quad 100.$$

```

for K in [5, 20, 100]:

    k, C = compute_fs_coefficients(
        t, x, T0, K
    )

    x_rec = reconstruct_fs(
        t, k, C, T0
    )

    plt.plot(
        t,
        np.real(x_rec),
        label=f"K={K}"
    )

```

The important observation is that increasing K improves the approximation away from discontinuities, while oscillatory behavior remains near jumps.

19 22. Magnitude and Phase of FS Coefficients

For

$$C_k = |C_k|e^{j\phi_k},$$

calculate

$$|C_k| = \text{np.abs}(C)$$

and

$$\phi_k = \text{np.angle}(C).$$

```
magnitude = np.abs(C)
phase = np.angle(C)

plt.stem(k, magnitude)
plt.xlabel("Harmonic k")
plt.ylabel("|C_k|")
plt.grid(True)
plt.show()

plt.stem(k, phase)
plt.xlabel("Harmonic k")
plt.ylabel("Phase")
plt.grid(True)
plt.show()
```

20 23. Fundamental Frequency and Harmonics

The fundamental frequency is

$$f_0 = \frac{1}{T_0},$$

and the k th harmonic is

$$f_k = kf_0.$$

Therefore:

```
f0 = 1 / T0
harmonic_frequencies = k * f0

plt.stem(
    harmonic_frequencies,
    np.abs(C)
)

plt.xlabel("Frequency (Hz)")
plt.ylabel("|C_k|")
plt.grid(True)
plt.show()
```

21 24. Trigonometric Fourier Coefficients

If a problem explicitly asks for a_k and b_k :

$$a_k = \frac{2}{T_0} \int x(t) \cos(k\omega_0 t) dt,$$

$$b_k = \frac{2}{T_0} \int x(t) \sin(k\omega_0 t) dt.$$

Implementation:

```
def compute_trig_coefficients(t, x, T0, K):  
  
    omega0 = 2 * np.pi / T0  
  
    a = np.zeros(K + 1)  
    b = np.zeros(K + 1)  
  
    a[0] = (  
        2 * np.trapezoid(x, t)  
        / T0  
    )  
  
    for k in range(1, K + 1):  
  
        a[k] = (  
            2 / T0  
            * np.trapezoid(  
                x * np.cos(k * omega0 * t),  
                t  
            )  
        )  
  
        b[k] = (  
            2 / T0  
            * np.trapezoid(  
                x * np.sin(k * omega0 * t),  
                t  
            )  
        )  
  
    return a, b
```

Then verify

$$C_k = \frac{1}{2}(a_k - jb_k), \quad k > 0.$$

22 25. General Problem-Solving Template

Almost every Fourier Series property problem can be solved using this structure.

1. Determine T_0 .

2. Calculate

$$\omega_0 = \frac{2\pi}{T_0}.$$

3. Create one or more periods of the time grid.

4. Define the original signal.
5. Perform the requested time-domain operation.
6. Directly calculate the new FS coefficients.
7. Independently calculate the original coefficients.
8. Apply the relevant FS property.
9. Compare actual and theoretical coefficients.
10. Plot coefficient magnitude and phase if required.
11. If reconstruction is requested, sum the finite series.

23 26. Comparing Complex Coefficients

For two coefficient arrays C_{actual} and C_{theory} , the simplest error is

$$\text{MSE}_{\text{complex}} = \frac{1}{N} \sum_k |C_{\text{actual},k} - C_{\text{theory},k}|^2.$$

Python:

```
mse_complex = np.mean(
    np.abs(
        C_actual - C_theory
    )**2
)
```

Magnitude MSE:

```
mse_mag = np.mean(
    (
        np.abs(C_actual)
        - np.abs(C_theory)
    )**2
)
```

Wrapped phase MSE:

```
phase_diff = (
    np.angle(C_actual)
    - np.angle(C_theory)
    + np.pi
) % (2 * np.pi) - np.pi

mse_phase = np.mean(
    phase_diff**2
)
```

For Fourier Series coefficients, phase at coefficients with nearly zero magnitude is unreliable, so a magnitude mask can be used before calculating phase MSE.

24 27. CFT vs Fourier Series: Important Coding Difference

For a CFT,

$$X(f) = \int_{-\infty}^{\infty} x(t)e^{-j2\pi ft} dt,$$

and f is continuously sampled over a frequency grid.

For Fourier Series,

$$C_k = \frac{1}{T_0} \int_{t_0}^{t_0+T_0} x(t)e^{-jk\omega_0 t} dt,$$

and only discrete harmonic indices are calculated:

$$k = 0, \pm 1, \pm 2, \dots$$

Thus a CFT program commonly contains:

```
f = np.linspace(f_min, f_max, M)
X = compute_cft(t, x, f)
```

whereas an FS program contains:

```
k = np.arange(-K, K + 1)
k, C = compute_fs_coefficients(t, x, T0, K)
```

The Fourier Series coefficients are therefore a **discrete sequence indexed by harmonic number k** , not a continuous spectrum.

25 28. Most Important Problems to Practice

For a programming exam, prioritize:

1. Numerical calculation of C_k .
2. Fourier Series reconstruction.
3. Linearity.
4. Time shifting.
5. Time reversal.
6. Real-signal conjugate symmetry.
7. Even/odd symmetry.
8. Differentiation.
9. Integration.
10. Complex exponential modulation.
11. Cosine modulation.
12. Sine modulation.
13. Multiplication of periodic signals.

14. Periodic convolution.
15. Parseval's theorem.
16. DC coefficient verification.
17. Gibbs phenomenon.

29. Final Mental Model

For a Fourier Series property-verification problem, always use two independent routes.

Route 1: Direct

$$y(t) \longrightarrow \text{calculate } Y_k \text{ directly}$$

using

$$Y_k = \frac{1}{T_0} \int y(t) e^{-jk\omega_0 t} dt.$$

Route 2: Property

$$C_k \longrightarrow \text{apply the property} \longrightarrow Y_k^{\text{theory}}$$

Then compare

$$Y_k^{\text{actual}} \quad \text{vs} \quad Y_k^{\text{theory}}.$$

For reconstruction:

$$C_k \longrightarrow \sum_{k=-K}^K C_k e^{jk\omega_0 t} \longrightarrow x_K(t).$$

The three formulas worth memorizing for implementation are

$$C_k = \frac{1}{T_0} \int_{t_0}^{t_0+T_0} x(t) e^{-jk\omega_0 t} dt$$

$$x_K(t) = \sum_{k=-K}^K C_k e^{jk\omega_0 t}$$

and

$$\text{verification} = \text{direct coefficients} \quad \text{vs} \quad \text{property-predicted coefficients.}$$